Create, Read, Update, and Delete with SQLAlchemy

FORK (https://github.com/learn-co-curriculum/python-p3-crud-with-sqlalchemy/fork) (https://github.com/learn-co-curriculum/python-p3-crud-with-sqlalchemy) (https://github.com/learn-co-curriculum/python-p3-crud-with-sqlalchemy/issues/new)

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
- Use SQLAlchemy to create, read, update and delete records in a SQL database.

Key Vocab

- **Schema**: the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist**: save a schema in a database.
- **Engine**: a Python object that translates SQL to Python and vice-versa.
- **Session**: a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction**: a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration**: the process of moving data from one or more databases to one or more target databases.

Introduction

In the previous lesson, we created and persisted a database schema using SQLAlchemy. This required us to define classes that inherited from a common `declarative_base` object and that possessed certain attributes that would be used to assign a table name, columns, primary keys, and more.

In this lesson, we'll be building on the same schema. The code from last lesson's code-along can be found in `app/sqlalchemy_sandbox.py`. Run `chmod +x app/sqlalchemy_sandbox.py` to make it executable.

> **Note**: we are using a SQLite database in memory now instead of a `students.db` file. This will allow us to make changes to our schema without running into issues. We will learn how to address changing a schema when we discuss **migrations** later in this module.

The Session

SQLAlchemy interacts with the database through **sessions**. These wrap **engine** objects like the one we included in our script in `sqlalchemy_sandbox.py`. The session contains an **identity map**, which is similar to an empty dictionary with keys for the table name, columns, and primary keys. When the session pulls data from `students.db`, it fills the identity map and uses it to populate a `Student` object with specific attribute values. When it commits data to the database, it fills the identity map in the same fashion but unpacks it into a `students` row instead.

`sessionmaker`

To create a session, we need to use SQLAlchemy's `sessionmaker` class. This ensures that there is a consistent identity map for the duration of our session.

Let's create a session in `sqlalchemy_sandbox.py` so that we can start executing statements in `students.db`:

```
# app/sqlalchemy_sandbox.py

#!/usr/bin/env python3

from datetime import datetime

from sqlalchemy import (create_engine, desc,
    CheckConstraint, PrimaryKeyConstraint, UniqueConstraint,
    Index, Column, DateTime, Integer, String)
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()
```

```
class Student(Base):
    __tablename__ = 'students'

    id = Column(Integer(), primary_key=True)
    name = Column(String())

if __name__ == '__main__':
    engine = create_engine('sqlite:///:memory:')
    Base.metadata.create_all(engine)

    # use our engine to configure a 'Session' class
    Session = sessionmaker(bind=engine)
    # use 'Session' class to create 'session' object
    session = Session()
```

Run `app/sqlalchemy_sandbox.py` to persist your schema and create a session. You won't see anything yet, but it's always wise to stop and check for errors after you change the functionality of your code.

Transactions

Transactions are a strategy for executing SQL statements via ORM that ensure that they all succeed or fail as a group. This is especially important if statements that occur later on depend on earlier statements executing properly. The workflow for a transaction is illustrated in the image below:

If any of the SQL statements in the above image fail to execute properly, the database will be rolled back to the state recorded at the beginning of the transaction and the process will end, returning an error message.

Refactoring our Schema

Before we begin transactions on our database, let's take a moment to build upon the `Student` model from the previous lesson:

```
#!/usr/bin/env python3

# imports

class Student(Base):
    __tablename__ = 'students'
    __table_args__ = (
        PrimaryKeyConstraint(
            'id',
            name='id_pk'),
        UniqueConstraint(
            'email',
            name='unique_email'),
        CheckConstraint(
            'grade BETWEEN 1 AND 12',
            name='grade_between_1_and_12')
    )

    Index('index_name', 'name')

    id = Column(Integer())
    name = Column(String())
    email = Column(String(55))
    grade = Column(Integer())
    birthday = Column(DateTime())
    enrolled_date = Column(DateTime(), default=datetime.now())

    def __repr__(self):
        return f"Student {self.id}: " \
            + f"{self.name}, " \
            + f"Grade {self.grade}"
```

```
# script
```

Let's break down some of the new features in the `Student` model:

Constraints

Along with keys, constraints help ensure that our data meets certain criteria before being stored in the database. Constraints are stored in the optional `__table_args__` class attribute. There are three main classes of constraint:

1. `PrimaryKeyConstraint` : assigns primary key status to a `Column` . This can also be accomplished through the optional `primary_key` argument to the `Column` class constructor.
2. `UniqueConstraint` : checks new records to ensure that they do not match existing records at unique `Column` s.
3. `CheckConstraint` : uses SQL statements to check if new values meet specific criteria.

Our new constraints for the `Student` model ensure that `id` is a primary key, `email` is unique, and `grade` is between 1 and 12.

Indexes

Indexes are used to speed up lookups on certain column values. Since teachers and administrators don't typically know their student's ID numbers off the top of their heads, it's wise to set up an index for `name` in preparation for people using it in their database transactions.

`__repr__()`

All classes in Python have a `__repr__()` instance method that determines their standard output value (i.e. what you see when you `print()` the object). By default, this shows the classname and an arbitrary ID. This default value is not very helpful in telling different objects apart. (At least not to humans.)

The `__repr__()` method in our refactored `Student` class will output a much more helpful string:

```
my_student = Student(...)
print(my_student)
# => Student 1: Joseph Smith, Grade 4
```

Input Sizes, Defaults, and More

SQLAlchemy provides a number of other optional arguments in the `Column` and data-type constructors that will allow you to make your code more specific and secure. Explore **the `Column` documentation (https://docs.sqlalchemy.org/en/14/core/sqlelement.html)** to learn more.

Creating Records

Now that we have a session and a robust data model, let's start populating a database.

To create a new student record in our database, we need to create an object using the `Student` class. This syntax is the same as with instantiating any other Python class.

> **Note**: while we can enter the data in order without argument names, we are going to use them consistently in class constructors when using SQLAlchemy. This is because it makes our code much more readable when working with tables with many columns.

```
# imports, models

if __name__ == '__main__':

    engine = create_engine('sqlite:///:memory:')
    Base.metadata.create_all(engine)

    Session = sessionmaker(bind=engine)
    session = Session()

    albert_einstein = Student(
        name="Albert Einstein",
        email="albert.einstein@zurich.edu",
        grade=6,
        birthday=datetime(
            year=1879,
```

```
            month=3,
            day=14
        ),
    )

    session.add(albert_einstein)
    session.commit()

    print(f"New student ID is {albert_einstein.id}.")
```

Now when we run our script from the command line, we see the following:

```
$ python app/sqlalchemy_sandbox.py
# => New student ID is 1.
```

After creating a `Student` object, `session.add()` generates a statement to include in the session's transaction, then `session.commit()` executes all statements in the transaction and saves any changes to the database. `session.commit()` will also update your `Student` object with a `id`.

If we want to save multiple new records in a single line of code, we can use the session's `bulk_save_objects()` instance method:

```
# imports, models

if __name__ == '__main__':

    engine = create_engine('sqlite:///:memory:')
    Base.metadata.create_all(engine)

    Session = sessionmaker(bind=engine)
    session = Session()

    albert_einstein = Student(
```

```
        name="Albert Einstein",
        email="albert.einstein@zurich.edu",
        grade=6,
        birthday=datetime(
            year=1879,
            month=3,
            day=14
        ),
    )

    alan_turing = Student(
        name="Alan Turing",
        email="alan.turing@sherborne.edu",
        grade=11,
        birthday=datetime(
            year=1912,
            month=6,
            day=23
        ),
    )

    session.bulk_save_objects([albert_einstein, alan_turing])
    session.commit()

    print(f"New student ID is {albert_einstein.id}.")
    print(f"New student ID is {alan_turing.id}.")
```

Let's run the script again to see what we've got:

```
$ python app/sqlalchemy_sandbox.py
# => New student ID is None.
```

```
# => New student ID is None.
```

Unfortunately, `bulk_save_objects()` does not associate the records with the session, so we don't update our records' IDs. Take this into consideration when creating records in your own code.

Run `app/sqlalchemy_sandbox.py` to make sure that there are no errors in your code. Once you're seeing the same output as above, let's practice retrieving these new records from the database.

Read Records

There are many ways to structure a query in SQLAlchemy, but they all begin with the session's `query()` instance method:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    session.bulk_save_objects([albert_einstein, alan_turing])
    session.commit()

    students = session.query(Student)

    print([student for student in students])

# => [Student 1: Albert Einstein, Grade 6, Student 2: Alan Turing, Grade 11]
```

We would see the same output using the `all()` instance method:

```
# imports, models
```

```
if __name__ == '__main__':

    # create session, student objects

    session.bulk_save_objects([albert_einstein, alan_turing])
    session.commit()

    students = session.query(Student).all()

    print(students)

# => [Student 1: Albert Einstein, Grade 6, Student 2: Alan Turing, Grade 11]
```

▶ *Which method helps make an object's standard output human-readable?*

Selecting Only Certain Columns

By default, the `query()` method returns complete records from the data model passed in as an argument. If we're only looking for certain fields, we can specify this in the arguments we pass to `query()`. Here's how we would retrieve all of the students' names:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    names = [name for name in session.query(Student.name)]

    print(names)

# => [('Albert Einstein',), ('Alan Turing',)]
```

Ordering

By default, results from any database query are ordered by their primary key. The `order_by()` method allows us to sort by any column:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    students_by_name = [student for student in session.query(
            Student.name).order_by(
            Student.name)]

    print(students_by_name)

# => [('Alan Turing',), ('Albert Einstein',)]
```

▶ *What data type does `query()` return records in?*

To sort results in descending order, we need to use the `desc()` function from the `sqlalchemy` module:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    students_by_grade_desc = [student for student in session.query(
            Student.name, Student.grade).order_by(
            desc(Student.grade))]
```

```
    print(students_by_grade_desc)

# => [('Alan Turing', 11), ('Albert Einstein', 6)]
```

Limiting

To limit your result set to the first `x` records, you can use the `limit()` method:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    oldest_student = [student for student in session.query(
            Student.name, Student.birthday).order_by(
            desc(Student.grade)).limit(1)]

    print(oldest_student)

# => [('Alan Turing', datetime.datetime(1912, 6, 23, 0, 0))]
```

The `first()` method is a quick and easy way to execute a `limit(1)` statement and does not require a list interpretation:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    oldest_student = session.query(
            Student.name, Student.birthday).order_by(
```

```
            desc(Student.grade)).first()

    print(oldest_student)

# => ('Alan Turing', datetime.datetime(1912, 6, 23, 0, 0))
```

func

Importing `func` from `sqlalchemy` gives us access to common SQL operations through functions like `sum()` and `count()`. As these operations act upon columns, we carry them out through wrapping a `Column` object passed to the `query()` method:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    student_count = session.query(func.count(Student.id)).first()

    print(student_count)

# => (2,)
```

It is best practice to call these functions as `func.operation()` rather than their name alone because many of these functions have name conflicts with functions in the Python standard library, such as `sum()`.

Filtering

Retrieving specific records requires use of the `filter()` method. A typical `filter()` statement has a column, a standard operator, and a value. It is possible to chain multiple `filter()` statements together, though it is typically easier to read with comma-separated clauses inside of one `filter()` statement.

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    query = session.query(Student).filter(Student.name.like('%Alan%'),
        Student.grade == 11)

    for record in query:
        print(record.name)

# => Alan Turing
```

Updating Data

There are several ways to update data using SQLAlchemy ORM. The simplest is to use Python to modify objects directly and then commit those changes through the session. For instance, let's say that a new school year is starting and our students all need to be moved up a grade:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    for student in session.query(Student):
        student.grade += 1

    session.commit()
```

```
    print([(student.name,
        student.grade) for student in session.query(Student)])

# => [('Albert Einstein', 7), ('Alan Turing', 12)]
```

The `update()` method allows us to update records without creating objects beforehand. Here's how we would carry out the same statement with `update()`:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    session.query(Student).update({
        Student.grade: Student.grade + 1
    })

    print([(
        student.name,
        student.grade
    ) for student in session.query(Student)])

# => [('Albert Einstein', 7), ('Alan Turing', 12)]
```

> **Note**: chaining of filters and query methods is possible because each method returns a modified version of the original object. Though certain attributes have changed, the returned object still has access to the same methods as the original object.

Deleting Data

To delete a record from your database, you can use the `delete()` method. If you have an object in memory that you want to delete, you can call the `delete()` method on the object from your `session`:

```
# imports, models

if __name__ == '__main__':

    # create session, student objects

    query = session.query(
        Student).filter(
            Student.name == "Albert Einstein")

    # retrieve first matching record as object
    albert_einstein = query.first()

    # delete record
    session.delete(albert_einstein)
    session.commit()

    # try to retrieve deleted record
    albert_einstein = query.first()

    print(albert_einstein)

# => None
```

If you don't have a single object ready for deletion but you know the criteria for deletion, you can call the `delete()` method from your query instead:

```
# imports, models
```

```
if __name__ == '__main__':

    # create session, student objects

    query = session.query(
        Student).filter(
            Student.name == "Albert Einstein")

    query.delete()

    albert_einstein = query.first()

    print(albert_einstein)

# => None
```

This strategy will delete all records returned by your query, so be careful!

Conclusion

This has been a *very* long lesson, but hopefully you've gotten some good practice performing CRUD on databases with SQLAlchemy. Remember that **sessions** allow us to interact with databases through SQLAlchemy and that those interactions are grouped into **transactions**. There are hundreds of methods, operations, and filter criteria available in SQLAlchemy, so make sure to keep the **SQLAlchemy ORM documentation (https://docs.sqlalchemy.org/en/14/orm/)** nearby as you finish up Phase 3!

Solution Code

```
# app/sqlalchemy_sandbox.py
```

```
from datetime import datetime

from sqlalchemy import (create_engine, desc,
    CheckConstraint, PrimaryKeyConstraint, UniqueConstraint,
    Index, Column, DateTime, Integer, String)
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'
    __table_args__ = (
        PrimaryKeyConstraint(
            'id',
            name='id_pk'),
        UniqueConstraint(
            'email',
            name='unique_email'),
        CheckConstraint(
            'grade BETWEEN 1 AND 12',
            name='grade_between_1_and_12'))

    Index('index_name', 'name')

    id = Column(Integer())
    name = Column(String())
    email = Column(String(55))
    grade = Column(Integer())
    birthday = Column(DateTime())
    enrolled_date = Column(DateTime(), default=datetime.now())

    def __repr__(self):
```

```
        return f"Student {self.id}: " \
            + f"{self.name}, " \
            + f"Grade {self.grade}"

if __name__ == '__main__':

    engine = create_engine('sqlite:///:memory:')
    Base.metadata.create_all(engine)

    Session = sessionmaker(bind=engine)
    session = Session()

    albert_einstein = Student(
        student_name="Albert Einstein",
        student_email="albert.einstein@zurich.edu",
        student_grade=6,
        student_birthday=datetime(
            year=1879,
            month=3,
            day=14
        ),
    )

    alan_turing = Student(
        student_name="Alan Turing",
        student_email="alan.turing@sherborne.edu",
        student_grade=11,
        student_birthday=datetime(
            year=1912,
            month=6,
            day=23
        ),
    )
```

```
session.bulk_save_objects([albert_einstein, alan_turing])
session.commit()
```

Resources

- SQLAlchemy ORM Documentation (https://docs.sqlalchemy.org/en/14/orm/)
- SQLAlchemy ORM Session Basics (https://docs.sqlalchemy.org/en/14/orm/session_basics.html)
- SQLAlchemy ORM Column Elements and Expressions (https://docs.sqlalchemy.org/en/14/core/sqlelement.html)
- SQLAlchemy ORM Querying Guide (https://docs.sqlalchemy.org/en/14/orm/queryguide.html)